

Teach Data Preprocessing

You are my **Machine Learning Mentor, Data Scientist Interview Trainer, and Case Study Instructor.**

I am following a **60-day / 2-month Machine Learning roadmap** to become **job-ready for a Data Scientist role** And its Third day and teach me according to it

Cover topic such as theory mathematical intuition ,advantages ,Disadvantages , applications ,python code practice ,1project ,30 mcq questions and Potential interview questions

Show moreShow less

Absolutely. Since **Day 1 covered the fundamentals of ML, AI vs ML vs DL, types of ML, supervised/unsupervised learning, regression/classification, features, targets, samples, and observations**, we'll move to the next logical foundation.

Machine Learning — Day 3

Topic: Data Preprocessing & Exploratory Data Analysis (EDA)

This is one of the **most important skills for a Data Scientist** because real-world ML performance depends heavily on how well you prepare and understand the data.

Today's objectives

By the end of Day 3, you should be able to:

1. Understand the ML data pipeline
2. Identify numerical and categorical variables
3. Handle missing values
4. Detect and treat outliers
5. Understand duplicates
6. Encode categorical variables
7. Understand feature scaling
8. Understand normalization vs standardization
9. Perform train/test splitting correctly
10. Understand data leakage
11. Perform basic EDA
12. Build a complete preprocessing pipeline using Python
13. Solve a real-world case study
14. Answer Data Scientist interview questions

15. Test yourself with 30 MCQs

1. What is Data Preprocessing?

Raw data is rarely ready for machine learning.

For example:

Age	Salary	City	Experience	Purchased
25	40000	Mumbai	2	Yes
31	65000	Pune	5	No
NaN	55000	Mumbai	4	Yes
29	NaN	Delhi	3	No
500	60000	Pune	4	Yes

Problems:

- Missing values
- Possible outliers
- Categorical variables
- Different scales
- Potential duplicate records
- Potential data-quality problems

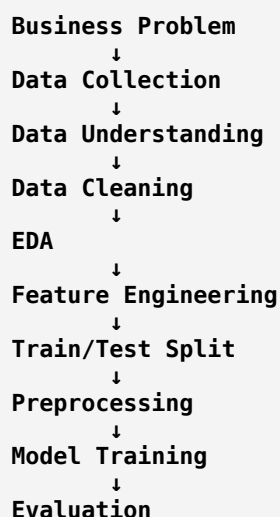
ML algorithms generally require **structured, numerical, meaningful input**.

Therefore:

Raw Data → Cleaning → Transformation → Feature Engineering → Model

2. Data Science Pipeline

A typical Data Scientist workflow:



↓
Deployment
↓
Monitoring

A common beginner mistake is:

Data → Model

A professional workflow is:

Business Problem
↓
Understand Data
↓
Clean Data
↓
EDA
↓
Prepare Features
↓
Train Model

3. Types of Data

A. Numerical Data

Numbers that represent quantities.

Examples:

Age = 28
Salary = 75000
Credit Score = 720
Balance = 150000

Two major types:

Continuous

Can take many values.

Examples:

Height = 175.4 cm
Weight = 61.8 kg
Temperature = 32.7°C

Discrete

Usually countable values.

Number of transactions = 10
Number of children = 2
Number of complaints = 4

4. Categorical Data

Represents categories.

Example:

```
Gender = Male/Female
City = Mumbai/Pune/Delhi
Education = Graduate/Postgraduate
Payment = Credit Card/Cash/UPI
```

Two important types:

Nominal

No natural order.

```
Mumbai
Pune
Delhi
```

Ordinal

Has an order.

```
Low
Medium
High
```

This distinction becomes important when encoding variables.

5. Missing Values

Suppose:

```
Age
25
31
NaN
29
35
```

NaN means the value is missing.

Why missing values matter

Many ML algorithms cannot directly handle missing values.

For example:

```
Python
model.fit(X, y)
```

may fail if x contains NaN values, depending on the algorithm.

6. How to Detect Missing Values

Using Pandas:

```
Python
df.isnull()
```

Count missing values:

```
Python
df.isnull().sum()
```

Percentage:

```
Python
df.isnull().mean() * 100
```

Example:

```
Python
import pandas as pd

data = {
    "Age": [25, 31, None, 29, 35],
    "Salary": [40000, 65000, 55000, None, 70000]
}

df = pd.DataFrame(data)
print(df.isnull().sum())
```

Output:

```
Age      1
Salary   1
```

7. Handling Missing Values

There are several approaches.

Method 1 — Delete rows

```
Python
df.dropna()
```

Useful when only a very small number of observations are missing and removing them won't bias the dataset.

Method 2 — Mean Imputation

Suppose:

```
Age = 25, 30, 35, NaN, 40
```

Mean:

$$\mu = \frac{25 + 30 + 35 + 40}{4}$$

$$\mu = 32.5$$

Replace:

```
NaN → 32.5
```

Python:

```
Python
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

Problem

Mean is sensitive to outliers.

8. Median Imputation

If the data contains outliers, median is often more robust.

Example:

```
20
25
30
35
500
```

Mean is heavily influenced by 500.

Median remains much more representative.

```
Python
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Interview rule

Mean → relatively symmetric numerical distribution

Median → skewed distribution / outliers

9. Mode Imputation

For categorical variables:

```
Python
df["City"] = df["City"].fillna(df["City"].mode()[0])
```

Example:

```
Mumbai
Mumbai
Pune
Mumbai
NaN
```

Mode:

```
Mumbai
```

So:

```
NaN → Mumbai
```

10. Advanced Imputation

More sophisticated methods include:

KNN Imputation

Uses similar observations.

```
Python
from sklearn.impute import KNNImputer
```

Iterative Imputation

Predicts missing values using other variables.

```
Python
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
```

These can be useful, but don't automatically make them your default. Simple imputation is often a strong baseline.

11. Outliers

An outlier is an observation that is unusually far from the majority of the data.

Example:

Salary:

30,000
35,000
40,000
42,000
45,000
2,00,00,000

The last value is suspicious/extreme.

But remember:

An outlier is not automatically an error.

A billionaire's salary isn't necessarily a data-entry mistake.

12. IQR Method

The Interquartile Range:

$$IQR = Q_3 - Q_1$$

Lower boundary:

$$Q_1 - 1.5(IQR)$$

Upper boundary:

$$Q_3 + 1.5(IQR)$$

Anything outside these boundaries is commonly flagged as an outlier.

Python:

```
Python
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[
    (df["Salary"] < lower) |
    (df["Salary"] > upper)
]

print(outliers)
```

13. Z-Score

Another approach:

$$Z = \frac{x - \mu}{\sigma}$$

Where:

- x = observation
- μ = mean
- σ = standard deviation

A common rule of thumb:

```
|Z| > 3 → potential outlier
```

But this assumption works best when the distribution is reasonably compatible with the method; it isn't a universal law.

14. Duplicates

Check duplicates:

```
Python
df.duplicated()
```

Count:

```
Python
df.duplicated().sum()
```

Remove:

```
Python
df.drop_duplicates(inplace=True)
```

Example:

Customer_ID	Age	Salary
101	25	40000
102	30	50000
101	25	40000

The third row might be a duplicate.

15. Encoding Categorical Variables

ML models generally need numerical representations.

Suppose:

```
City
Mumbai
Pune
```

Delhi
Mumbai

We need to transform it.

16. Label Encoding

Example:

Low → 0
Medium → 1
High → 2

This makes sense when categories have an order.

```
Python
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df["Risk"] = le.fit_transform(df["Risk"])
```

Warning

Don't blindly label-encode nominal categories.

If:

Mumbai → 0
Pune → 1
Delhi → 2

the numbers may falsely imply:

Delhi > Pune > Mumbai

There is no such relationship.

17. One-Hot Encoding

For nominal categories:

City
Mumbai
Pune
Delhi

becomes:

	Mumbai	Pune	Delhi
	1	0	0
	0	1	0

Mumbai	Pune	Delhi
0	0	1

Python:

```
Python
pd.get_dummies(df, columns=["City"])
```

Or with scikit-learn:

```
Python
from sklearn.preprocessing import OneHotEncoder
```

18. Feature Scaling

Suppose we have:

```
Age = 30
Salary = 80000
```

These features have very different scales.

Some algorithms can be strongly affected by this.

Scaling transforms features into comparable numerical ranges.

19. Standardization

Formula:

$$z = \frac{x - \mu}{\sigma}$$

After standardization:

```
Mean ≈ 0
Standard deviation ≈ 1
```

Python:

```
Python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

20. Normalization

Min-Max scaling:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Usually transforms values to:

0 → 1

Python:

```
Python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)
```

21. Standardization vs Normalization

Standardization	Min-Max Normalization
Mean $\approx$ 0	Range usually 0–1
SD $\approx$ 1	Uses min/max
Formula uses mean & SD	Formula uses min & max
Useful for many ML algorithms	Useful when bounded range is desirable
Less directly affected by extreme min/max than min-max	Sensitive to extreme values

22. Which Algorithms Need Scaling?

Scaling is particularly important for algorithms based on **distance, magnitude, gradients, or regularization**.

Examples:

- KNN
- K-Means
- SVM
- Logistic Regression
- Linear Regression with regularization
- Neural Networks
- PCA

Tree-based models generally don't require feature scaling:

- Decision Tree
- Random Forest
- XGBoost

- LightGBM

23. Train-Test Split

Suppose:

1000 observations

We might use:

80% → Training
20% → Testing

Python:

```
Python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

24. Why Split Data?

The model should learn from one dataset and be evaluated on **unseen data**.

```
Training Data
  ↓
Learn patterns
  ↓
Model
  ↓
Test Data
  ↓
Evaluate generalization
```

25. Data Leakage — VERY IMPORTANT

One of the most important interview concepts.

Suppose you calculate:

```
Python
scaler.fit_transform(X)
```

on the **entire dataset before splitting**.

The scaler has now seen information from the future test set.

That can create leakage.

Correct approach:

```
Split
↓
Fit preprocessing on training data
↓
Transform training data
↓
Transform test data using the same fitted transformation
```

Example:

```
Python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Even better: use a Pipeline.

26. EDA — Exploratory Data Analysis

EDA means:

Understanding your dataset before building a model.

Important questions:

```
How many rows?
How many columns?
What are the data types?
Any missing values?
Any duplicates?
Any outliers?
What is the target distribution?
Which variables correlate with the target?
Are there suspicious values?
Are classes imbalanced?
```

27. Essential Pandas EDA Commands

```
Python
df.head()
```

```
Python
df.tail()
```

```
Python
df.shape
```

```
Python
df.info()
```

```
Python
df.describe()
```

```
Python
df.nunique()
```

```
Python
df.isnull().sum()
```

```
Python
df.duplicated().sum()
```

Categorical distribution:

```
Python
df["City"].value_counts()
```

Correlation:

```
Python
df.corr(numeric_only=True)
```

28. EDA Visualization

Using Matplotlib:

```
Python
import matplotlib.pyplot as plt

df["Age"].hist()

plt.xlabel("Age")
plt.ylabel("Frequency")
plt.title("Age Distribution")

plt.show()
```

Boxplot:

```
Python
df.boxplot(column="Salary")

plt.show()
```

Scatter plot:

```
Python
plt.scatter(df["Age"], df["Salary"])

plt.xlabel("Age")
plt.ylabel("Salary")

plt.show()
```

29. Mathematical Intuition: Mean

Mean:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Example:

10, 20, 30

$$\bar{x} = \frac{10 + 20 + 30}{3} = 20$$

Mean is sensitive to extreme values.

30. Median

Sort observations and find the middle value.

10, 20, 30, 40, 50

Median:

30

For an even number:

10, 20, 30, 40

$$Median = \frac{20 + 30}{2} = 25$$

Median is more robust to extreme values.

31. Variance

Variance measures dispersion.

Population variance:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

Sample variance typically uses:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

You will see the $n - 1$ form frequently in statistical estimation.

32. Standard Deviation

$$\sigma = \sqrt{\sigma^2}$$

It measures how spread out the observations are around the mean.

Example:

Dataset A:

48, 49, 50, 51, 52

Dataset B:

10, 30, 50, 70, 90

Both have:

Mean = 50

But Dataset B has much greater variance.

33. Correlation

Pearson correlation:

$$r = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$$

Range:

$-1 \leq r \leq +1$

Interpretation:

+1 → perfect positive linear relationship
 0 → no linear correlation
 -1 → perfect negative linear relationship

Example:

Experience ↑
 Salary ↑

Potential positive correlation.

But remember:

Correlation does not imply causation.

34. Important EDA Mistake

Don't blindly remove:

```
outliers
missing values
highly correlated variables
```

Every data-cleaning decision should be based on:

business context + statistical evidence + model objective.

DAY 3 MINI CASE STUDY

Customer Churn Prediction

Imagine a telecom company wants to predict:

Will a customer churn?

Dataset:

Age	Tenure	MonthlyCharges	Contract	Churn
25	2	80	Monthly	Yes
42	5	60	Annual	No
35	1	NaN	Monthly	Yes
50	8	45	Annual	No

Objective

Predict:

$$P(Churn = 1)$$

35. Step 1 — Load Data

```
Python
import pandas as pd

df = pd.read_csv("customer_churn.csv")
print(df.head())
```

36. Step 2 — Understand Data

```
Python
```

```
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

37. Step 3 — Separate Features and Target

```
Python
X = df.drop("Churn", axis=1)
y = df["Churn"]
```

38. Step 4 — Train/Test Split

```
Python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Why stratify=y?

If churn is imbalanced:

```
90% No
10% Yes
```

we want the train/test sets to preserve approximately similar class proportions.

39. Step 5 — Identify Columns

```
Python
numerical_features = [
    "Age",
    "Tenure",
    "MonthlyCharges"
]

categorical_features = [
    "Contract"
]
```

40. Step 6 — Build Preprocessing Pipeline

This is closer to **professional Data Scientist practice**.

```
Python
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

Numerical preprocessing:

```
Python
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

Categorical preprocessing:

```
Python
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

Combine:

```
Python
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numerical_features),
    ("cat", categorical_pipeline, categorical_features)
])
```

41. Add a Model

For example, Logistic Regression:

```
Python
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression())
])
```

Train:

```
Python
model.fit(X_train, y_train)
```

Predict:

```
Python
predictions = model.predict(X_test)
```

Evaluate:

```
Python
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", accuracy)
```

42. Why Pipeline Is Powerful

Without pipeline:

```
Imputation
↓
Encoding
↓
Scaling
↓
Model
```

You have to manually ensure every transformation is correctly fitted.

With Pipeline:

```
Raw Data
↓
Preprocessor
↓
Model
```

Advantages:

- Reduces leakage risk
- Cleaner code
- Reproducible
- Easier deployment
- Easier cross-validation
- Consistent train/test transformations

Day 3 Project

Project: Customer Churn Prediction

Business Problem

A telecom company wants to identify customers who are likely to leave.

Dataset

Use the **Telco Customer Churn dataset**.

Recommended columns:

```
customerID
gender
SeniorCitizen
```

```
Partner
Dependents
tenure
PhoneService
InternetService
Contract
PaymentMethod
MonthlyCharges
TotalCharges
Churn
```

Your tasks

Phase 1 — Data Understanding

1. Load dataset
2. Shape
3. Data types
4. Missing values
5. Duplicate records
6. Unique values

Phase 2 — EDA

Analyze:

```
Churn distribution
Contract vs Churn
Tenure vs Churn
Monthly Charges vs Churn
Internet Service vs Churn
Payment Method vs Churn
```

Phase 3 — Preprocessing

Perform:

```
Missing-value treatment
Categorical encoding
Numerical scaling
Train/test split
```

Phase 4 — Modeling

Start with:

```
Logistic Regression
Decision Tree
Random Forest
```

Phase 5 — Evaluation

Don't rely only on accuracy.

Calculate:

```
Python
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

Because churn datasets can be imbalanced.

Real Data Scientist Thinking

Suppose your model gives:

Accuracy = 92%

You might think:

Great model!

But imagine:

95% customers = No Churn
5% customers = Churn

A model predicting:

Everyone → No Churn

could get approximately:

95% accuracy

while being useless for identifying churners.

Therefore, Data Scientists ask:

What business problem are we optimizing for?

If the company wants to identify customers at risk of leaving, **recall for the churn class** can be particularly important.



Mathematical Intuition: Confusion Matrix

For binary classification:

	Actual Positive	Actual Negative
Predicted Positive	TP	FP
Predicted Negative	FN	TN

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Recall

$$Recall = \frac{TP}{TP + FN}$$

F1

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

Business Interpretation

For churn:

False Positive

Model says:

Customer will churn

but customer doesn't.

Company may waste money offering a retention discount.

False Negative

Model says:

Customer won't churn

but customer actually leaves.

Company loses the customer.

Depending on business costs, the second mistake may be more expensive.

That's why **model evaluation should be connected to business impact**.

30 MCQs — Day 3

1. What is the primary purpose of preprocessing?

- A. Increase dataset size
- B. Prepare data for analysis/modeling
- C. Delete all rows
- D. Train neural networks

Answer: B

2. Which Pandas function detects missing values?

- A. `isnull()`
- B. `missing()`
- C. `nullcheck()`
- D. `findna()`

Answer: A

3. Which measure is more robust to extreme outliers?

- A. Mean
- B. Median
- C. Variance
- D. Standard deviation

Answer: B

4. Which method is commonly used for categorical missing values?

- A. Mean
- B. Median
- C. Mode
- D. Variance

Answer: C

5. IQR is:

- A. $Q1 + Q3$
- B. $Q3 - Q1$
- C. $Q3 \times Q1$
- D. $Q1 / Q3$

Answer: B

6. Common IQR outlier threshold is:

- A. $0.5 \times \text{IQR}$
- B. $1 \times \text{IQR}$
- C. $1.5 \times \text{IQR}$
- D. $10 \times \text{IQR}$

Answer: C

7. Standardization generally produces:

- A. Mean 1, SD 0
- B. Mean 0, SD 1
- C. Mean 10, SD 10
- D. Range 0-100

Answer: B

8. Min-Max scaling generally transforms values into:

- A. -100 to 100
- B. 0 to 1
- C. 1 to 10
- D. 10 to 100

Answer: B

9. Which algorithm generally doesn't require scaling?

- A. KNN
- B. SVM
- C. Random Forest
- D. K-Means

Answer: C

10. One-hot encoding is mainly used for:

- A. Numerical variables
- B. Nominal categorical variables
- C. Missing values
- D. Outliers

Answer: B

11. Label encoding is especially appropriate when categories are:

- A. Random
- B. Ordinal
- C. Continuous
- D. Missing

Answer: B

12. Which function removes duplicates?

- A. `remove_duplicates()`
- B. `drop_duplicates()`
- C. `delete_duplicates()`
- D. `duplicate_remove()`

Answer: B

13. What does EDA mean?

- A. Estimated Data Algorithm
- B. Exploratory Data Analysis
- C. Experimental Data Algorithm
- D. External Data Analysis

Answer: B

14. Which function provides descriptive statistics?

- A. `df.stats()`
- B. `df.describe()`
- C. `df.summary()`
- D. `df.statistics()`

Answer: B

15. What does `df.shape` return?

- A. Data types
- B. Number of rows and columns
- C. Missing values
- D. Mean values

Answer: B

16. What is data leakage?

- A. Data compression
- B. Unintended information from outside the training process influencing the model
- C. Data deletion
- D. Dataset duplication

Answer: B

17. Scaling should generally be fitted on:

- A. Test data
- B. Training data
- C. Entire dataset before splitting
- D. Validation labels

Answer: B

18. Which method can be used to split data?

- A. `train_test_split()`
- B. `data_splitter()`
- C. `split_ml()`
- D. `train_data()`

Answer: A

19. What does `stratify=y` help preserve?

- A. Feature scaling
- B. Class proportions
- C. Missing values
- D. Feature names

Answer: B

20. Pearson correlation ranges between:

- A. 0 and 1
- B. -1 and +1

- C. -100 and 100
- D. 1 and 10

Answer: B

21. Correlation implies:

- A. Always causation
- B. A measure of association, not necessarily causation
- C. Guaranteed prediction
- D. No relationship

Answer: B

22. Which is an ordinal variable?

- A. City
- B. Blood group
- C. Satisfaction: Low/Medium/High
- D. Customer ID

Answer: C

23. Which is nominal?

- A. Low/Medium/High
- B. Small/Medium/Large when size is ordered
- C. Mumbai/Pune/Delhi
- D. Poor/Average/Excellent

Answer: C

24. What does StandardScaler perform?

- A. One-hot encoding
- B. Standardization
- C. Label encoding
- D. Imputation

Answer: B

25. Which class performs one-hot encoding?

- A. StandardScaler
- B. OneHotEncoder
- C. SimpleImputer
- D. KNNImputer

Answer: B

26. Which metric measures TP among predicted positives?

- A. Recall
- B. Precision
- C. Accuracy
- D. Specificity

Answer: B

27. Which metric measures TP among actual positives?

- A. Precision
- B. Recall
- C. Accuracy
- D. F1

Answer: B

28. Why is accuracy potentially misleading for imbalanced datasets?

- A. It ignores all predictions
- B. Majority class can dominate the score
- C. It cannot be calculated
- D. It always equals recall

Answer: B

29. Which technique combines preprocessing and model training?

- A. Pipeline
- B. Histogram
- C. Correlation
- D. Encoding

Answer: A

30. Which is usually the safest statement about outliers?

- A. Always remove them
- B. Always keep them
- C. Investigate them using statistical and business context
- D. Replace every outlier with zero

Answer: C

Day 3 Interview Questions

Q1. How would you handle missing values?

Strong interview answer:

I first analyze the percentage and pattern of missingness and understand whether the missingness has business meaning. For numerical variables, I may use median or mean imputation depending on the distribution and outliers. For categorical variables, mode or a separate "Unknown" category can be appropriate. For more complex situations, I may use KNN or iterative imputation. Importantly, I fit the imputation strategy only on training data to prevent data leakage.

Q2. Mean vs median — when would you use each?

I prefer mean when the numerical variable is reasonably symmetric and doesn't contain influential outliers. I prefer median when the distribution is skewed or contains outliers because median is more robust.

Q3. What is data leakage?

Data leakage occurs when information that would not be available at prediction time influences model training. For example, fitting a scaler or imputer on the entire dataset before train-test splitting can allow information from the test set to influence preprocessing. I prevent this by fitting preprocessing only on training data, preferably using a scikit-learn Pipeline.

Q4. Why is scaling necessary?

Scaling puts features on comparable numerical scales. It is particularly important for distance-based, gradient-based and regularized algorithms such as KNN, SVM, logistic

regression and neural networks. Tree-based algorithms generally don't require scaling.

Q5. StandardScaler vs MinMaxScaler?

StandardScaler transforms a feature using its mean and standard deviation, producing approximately zero mean and unit variance. MinMaxScaler maps values to a specified range, commonly 0 to 1. Min-max scaling can be strongly affected by extreme minimum or maximum values.

Q6. How do you detect outliers?

A strong answer:

I use multiple approaches depending on the distribution and business context, including boxplots, IQR, z-scores, percentile analysis and domain-specific thresholds. I don't automatically remove outliers because they may represent legitimate rare events.

Q7. Why shouldn't you scale before train-test split?

Because the scaling parameters:

$$\mu, \sigma$$

would be calculated using information from both training and test data.

That introduces information leakage.

Q8. Why use Pipeline?

Pipeline ensures that preprocessing and modeling steps are applied consistently. It also helps prevent leakage, simplifies cross-validation, improves reproducibility and makes deployment easier.

Q9. What is EDA?

EDA is the systematic exploration of data using statistical summaries and visualizations to understand distributions, missing values, outliers, relationships, class imbalance and potential data-quality issues before modeling.

Q10. What would you check before training a model?

My checklist:

- ✓ Target definition
- ✓ Data types
- ✓ Missing values
- ✓ Duplicates
- ✓ Outliers
- ✓ Class imbalance
- ✓ Feature distributions
- ✓ Feature-target relationships
- ✓ Leakage
- ✓ Train/test split
- ✓ Feature encoding
- ✓ Scaling requirements
- ✓ Baseline model

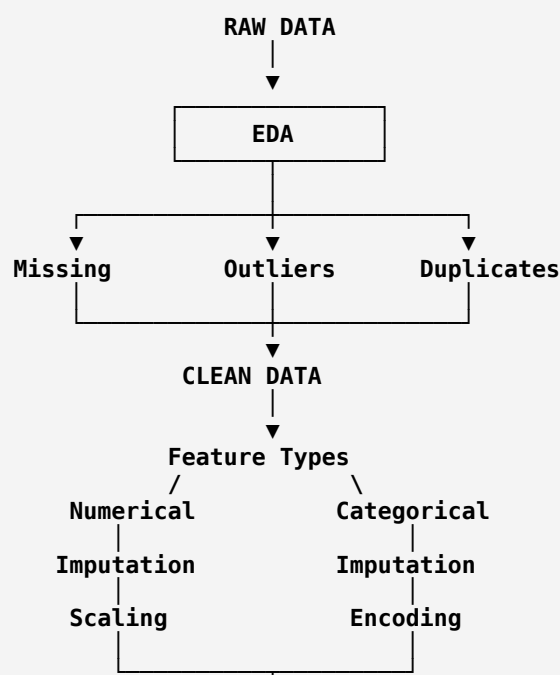
🏆 Day 3 Data Scientist Challenge

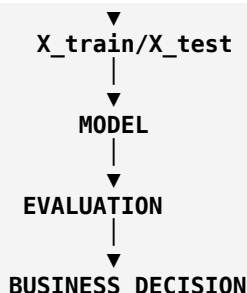
Before moving to Day 4, try to implement this yourself **without copying the solution**:

Dataset: Customer Churn

1. Load the dataset
2. Perform EDA
3. Find missing values
4. Find duplicates
5. Separate X and y
6. Split train/test
7. Identify numerical features
8. Identify categorical features
9. Build numerical preprocessing pipeline
10. Build categorical preprocessing pipeline
11. Combine them using ColumnTransformer
12. Train Logistic Regression
13. Calculate:
 - Accuracy
 - Precision
 - Recall
 - F1
14. Create confusion matrix
15. Explain which type of error is more expensive
16. Give 5 business recommendations

Your mental model for Day 3





🔥 Most important Day 3 concepts for interviews

If you remember only **10 things**, remember these:

1. **EDA comes before serious modeling.**
2. Mean is sensitive to outliers; median is more robust.
3. Don't automatically delete outliers.
4. Categorical variables require appropriate encoding.
5. Ordinal variables have meaningful order.
6. Scale features when the algorithm is sensitive to feature magnitude.
7. Tree-based algorithms generally don't need scaling.
8. **Always watch for data leakage.**
9. Fit preprocessing on training data, then transform validation/test data.
10. Use **Pipeline + ColumnTransformer** for robust preprocessing.

Day 4 should logically move from preprocessing into feature engineering and feature selection, including transformations, interaction features, binning, log transformations, multicollinearity, correlation-based selection, variance threshold, and practical feature-engineering exercises.

ChatGPT can make mistakes. Check important info.